



# ENGIN 604 INTRODUCTION TO PYTHON FOR FINANCE

## SEQUENCES & DATA STRUCTURES - SOLUTION

**Professor:** *Gabriel E. Cabrera*  
**Teaching Assistant:** *Alex Den Braber*



## 1 Basic Data Structures

As a general rule, data structures are objects that contain other objects (e.g. scalars) as sequences. Python provides the following *built-in* data structures:

Table 1: Data Structure Types

| Structure  | Example                  | Brackets | index | Repeated Elements | Mutable | Set Operations |
|------------|--------------------------|----------|-------|-------------------|---------|----------------|
| List       | A = [1,2,'hello']        | [ ]      | Yes   | Yes               | Yes     | No             |
| Tuple      | B = (3,3,5)              | ()       | Yes   | Yes               | No      | No             |
| Dictionary | D = {'Day':01,'Month':3} | { }      | Yes   | Yes               | Yes     | No             |
| Set        | C = {6,7,8}              | { }      | No    | No                | No      | Yes            |

### 1.1 List (`list`)

1. Build a list containing the names of the following technology companies: 'Facebook', 'Apple', 'Amazon' and 'Netflix'.
  - a. Print the first and last element of the list you created.

```
faan = ['Facebook', 'Apple', 'Amazon', 'Netflix']

# no. of elements in the list
len(faan)

# only the first element
faan[0]

# only the second element
faan[1]

# only the third element
faan[2]

# only the last element
faan[3]
```

```

# all the elements
faan[:]

# first element plus all the rest
faan[0:]

# second element plus all the rest
faan[1:]

# first element up to the third element (inclusive), the interval will be [,)
faan[0:3]

# select everything using an index that does not exist in the list but is larger than the existing
faan[0:1000]

```

- b. Print the first and last element of the list you created using negative indices.

```

# first element
faan[-4]

# last element
faan[-1]

```

- c. Add “Nvidia” at the beginning of the list. How would you add it in the fourth position?

```

faan.insert(0, 'Nvidia') # in-place
faan.insert(3, 'Nvidia') # fourth position (in-place)

```

- d. Add “Google” at the end of the list.

```

# important (in-place) by default it is the last position
faan.append('Google')

```

- e. How many elements does the list have so far?

```

len(faan)

```

- f. What position is the company ‘Apple’ at?

```

# it will show the first position that contains the element,
# but not all the ones that exist in the list
faan.index('Apple')

```

- g. Remove “Nvidia” and “Netflix” from the list.

```

# way 1
del faan[faan.index('Nvidia')] # it will show the first one
del faan[faan.index('Netflix')]

# way 2
faan.pop(faan.index('Nvidia')) # .pop() by default removes the last one
faan.pop(faan.index('Netflix'))

# way 3
faan.remove('Nvidia')
faan.remove('Netflix')

```

- h. Sort the list in descending order according to the number of characters of each element in the list.

```

faan.sort(key=len)

```

2. Build the following lists:

- `part_a = [[0, 'a'], [1, 'b']]`
- `part_b = [[2, 'c'], [3, 'd']]`

a. Concatenate or combine both lists. What is the difference between concatenating with `+` and with `extend`?

```
part_a = [[0, 'a'], [1, 'b']]
part_b = [[2, 'c'], [3, 'd']]

combined_list1 = part_a + part_b

# using extend
part_a.extend(part_b) # in-place
```

b. Print the second element of the list generated in (a).

```
# elements in the nested list
combined_list1[0]
combined_list1[0][0]
combined_list1[0][1]
```

3. Generate a list containing the numbers from 1 to 10.

a. Sort the list in descending order. What is the difference between sorting with `sort` and with `sorted`?

```
num_list = list(range(1, 11))

# sort
num_list.sort(reverse=True) # in-place

# sorted (built-in)
sorted(num_list)
```

b. Select the values whose index is even.

```
num_list[::2]
```

## 1.2 Tuple (tuple)

A tuple (tuple) is created if you use a comma (,) to separate values when assigning to a single variable:

```
tuple_ = 1, 2, 'Facebook', 'Amazon'
tuple_
```

They can even store lists (list):

```
nested_tuple2 = (1,2), ('Facebook', 'Amazon'), ['Apple', 'Netflix']
nested_tuple2
```

You can only modify an element of the tuple (tuple) that allows modification, for example a list (list):

```
nested_tuple2[2].append('Google')
nested_tuple2
```

It allows concatenation using the `+` sign:

```
('Facebook', 'Amazon') + ('Apple', 'Netflix')
```

And even multiplication of elements using `*`:

```
('Facebook', 'Amazon') * 3
```

To force a list (list) to become a tuple (tuple) it is enough to write:

```
tuple([1, 2, 3, 4])
```

### 1.3 Dictionary (dict)

Dictionaries (dict) are mutable data structures, characterized by the use of key-value pairs (*key-value*).

```
dict1 = {'Name' : 'Janet Yellen',  
        'Country' : 'United States',  
        'Profession' : ' United States secretary of the treasury',  
        'Age' : 74}  
  
type(dict1)
```

To access the values associated with a key, it is enough to use [ ] and write the name of the key:

```
print(dict1['Name'], dict1['Age'])
```

It inherits the .keys() method to access the names of the keys:

```
dict1.keys() # keys
```

It inherits the .values() method to access the values associated with the keys:

```
dict1.values() # values
```

With .items() you access both the key (*key*) and the values (*values*):

```
dict1.items() # items = keys + values
```

### 1.4 Set (set)

The set (set) data structure allows only unique values. To create a set (set), you create a list and then apply set():

```
set1 = set([1, 2, 3, 4, 5, 5])  
set1  
  
set2 = set([3, 4, 5, 6, 9, 9, 7])  
set2
```

We can apply set operations:

- All the items that are in set1 and set2:

```
# union  
set1.union(set2)
```

- Items that are in both set1 and set2

```
# intersection  
set1.intersection(set2)
```

- Items that are in set1 but not in set2:

```
set1.difference(set2)
```

- Items that are in set2 but not in set1:

```
set2.difference(set1)
```

- Items that are in set1 or set2 but not in both:

```
set1.symmetric_difference(set2)
```